Technical Documentation

Oct 16, 2026 · @Arnob

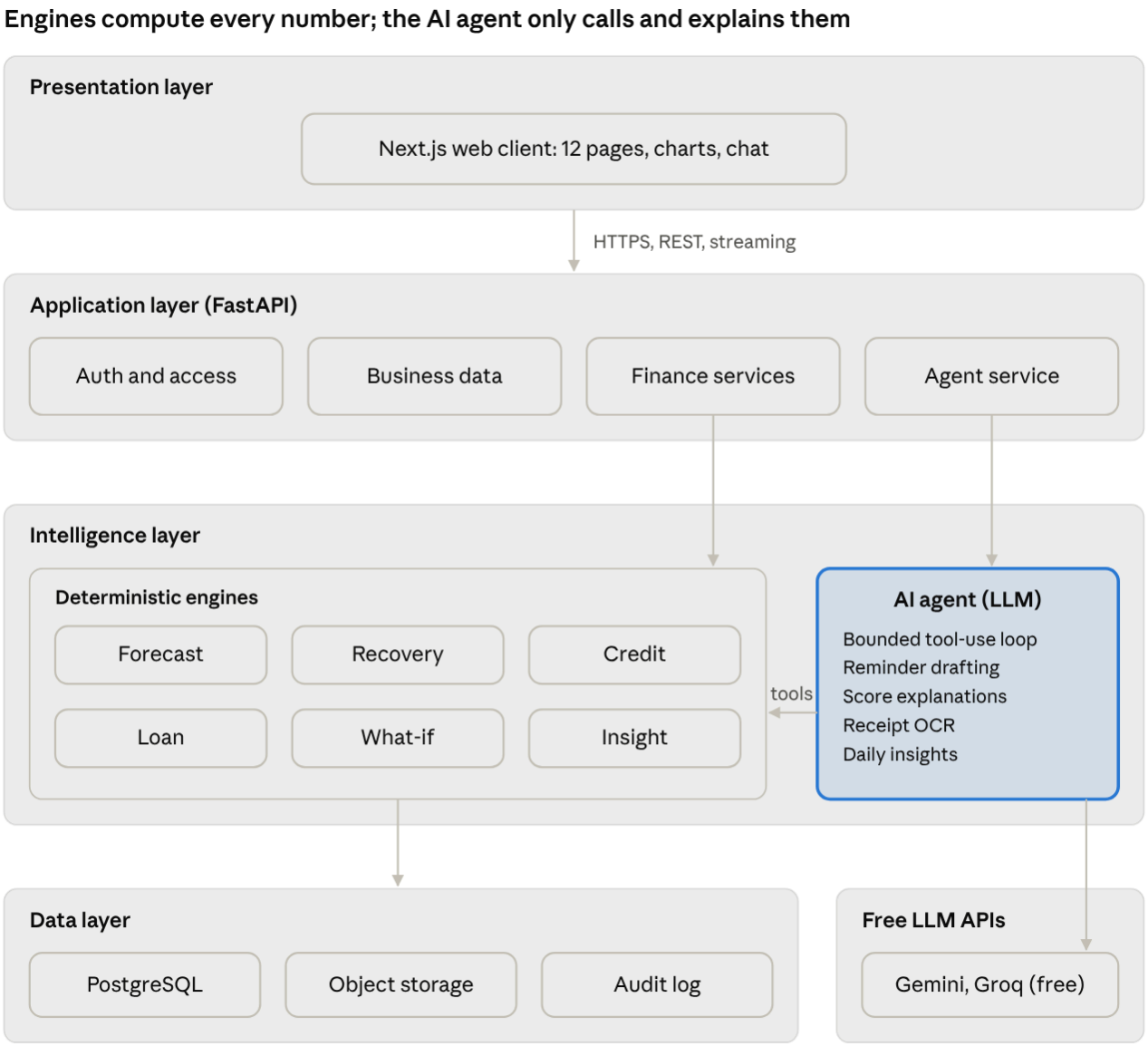
Introduction

The platform is an AI financial assistant for small and medium businesses. It warns owners about cash shortages 7 to 30 days in advance, tells them who to collect money from first, gives them an explainable credit score, and estimates how much they can safely borrow.

#	Feature	What it does
1	Cash Flow Prediction	Detects likely cash shortages over the next 7 to 30 days
2	Smart Payment Recovery	Ranks customers by outstanding dues and drafts payment reminders
3	AI Credit Scoring	Gives an explainable 300 to 900 score from income, cash flow, dues and repayment history
4	Loan Readiness & Recommendation	Estimates a safe loan amount and suitable financing options (not an approval)
5	AI Financial Agent	Answers questions in plain language using the four engines above as tools

1. System Architecture

The system has four layers: a Next.js web client, a FastAPI application server, an intelligence layer of deterministic engines plus an AI agent, and a PostgreSQL data layer.



System architecture · 4 layers

Requests flow top-down; the agent reaches data only by calling the engines as tools.

1.1 Design principle: deterministic core, AI interface

Every financial number is computed by deterministic, testable engines. The LLM never calculates figures; it interprets the user's intent, picks which engine to call, and explains the result. This keeps every output auditable and prevents hallucinated numbers.

No paid AI services. All AI features run on free options behind one LLM provider layer; switching provider is a single setting in the backend environment file. If a free-tier rate limit is hit, the request falls back to the next provider.

Order	Provider	Cost	Used for
1	Google Gemini API (free tier)	Free, rate-limited	Agent chat with tool calling, reminders, explanations, receipt reading
2	Groq API (free tier, open-source models)	Free, rate-limited	Fallback for chat and text generation
3	Ollama running an open model locally (e.g. Qwen 2.5 7B)	Free, runs offline	Offline fallback and demo backup
–	PaddleOCR or Tesseract	Free, open source	Receipt text extraction without any API
–	Templates	Free	Reminder and explanation text if every LLM provider is unavailable

1.2 Components

Component	Layer	Responsibility
Web client	Presentation	12 pages, charts, chat interface, form validation
API routes	Application	Request validation, authentication, rate limiting
Service layer	Application	Business logic and orchestration between engines and data
Forecast Engine	Intelligence	Projects daily cash balance 7 to 30 days ahead and flags shortages
Recovery Engine	Intelligence	Ranks overdue receivables by collection priority
Credit Engine	Intelligence	Computes an explainable score from 300 to 900
Loan Engine	Intelligence	Estimates safe borrowing capacity and matches financing products
What-if Engine	Intelligence	Re-runs the forecast under user-defined scenarios

Component	Layer	Responsibility
Insight Engine	Intelligence	Detects spending anomalies and period-over-period changes
AI Agent	Intelligence	Natural-language interface that calls the engines as tools
PostgreSQL + object storage	Data	Persistent data isolated per business; receipt and CSV files

1.3 Frontend pages

#	Page	Route	Contains
1	Landing	/	Product intro and call to action
2	Auth + Onboarding	/auth, /onboarding	Login, signup, business setup wizard
3	Dashboard	/dashboard	KPI cards, cash alerts, daily AI summary, notifications
4	Transactions	/transactions	List, add, CSV or mobile-wallet import, receipt scan
5	Cash Flow	/cashflow	Forecast chart, shortage days, calendar view
6	Ledger	/ledger	Receivables (invoices, recovery, reminders) and payables (suppliers, bills)
7	Customers	/customers	Customer list, profile, payment history, risk level
8	Credit Score	/credit-score	Score gauge, explanation factors, trend
9	Loan Center	/loan	Readiness, recommended amount, options, checklist, what-if simulator
10	Reports	/reports	P&L, expense insights, goals, PDF export
11	AI Agent	/agent	Chat, suggested questions, financial tips
12	Settings	/settings	Business profile, cash threshold, reminder tone, lender access

1.4 Data model

Every business-owned table carries a business_id, which is how tenant data is isolated.

Table	Key fields
users	id, name, phone, email, password_hash, created_at
businesses	id, user_id, name, type, started_at, cash_threshold
transactions	id, business_id, date, amount, type (income/expense), category, source (manual/csv/scan)
customers	id, business_id, name, phone, email, risk_level
invoices	id, customer_id, amount, issue_date, due_date, paid_date, status
reminders	id, invoice_id, message, tone, channel, sent_at
suppliers	id, business_id, name, phone
bills	id, supplier_id, amount, due_date, paid_date, status
loans	id, business_id, lender, principal, rate, tenure_months, start_date
repayments	id, loan_id, due_date, paid_date, amount
credit_scores	id, business_id, score, factors (JSON), calculated_at
loan_products	id, name, lender, min_amount, max_amount, rate, tenure, min_score
goals	id, business_id, title, target_amount, current_amount, deadline
notifications	id, business_id, type, message, is_read, created_at
chat_messages	id, business_id, role, content, created_at
lender_access	id, business_id, lender_name, scope, granted_at, expires_at
audit_log	id, business_id, actor, action, target, created_at

1.5 Main API endpoints

Module	Endpoints
Auth	POST /auth/signup, POST /auth/login, POST /onboarding
Dashboard	GET /dashboard/summary
Transactions	GET, POST /transactions; POST /transactions/import; POST /transactions/scan
Cash flow	GET /cashflow/forecast?days=30; GET /cashflow/calendar
Ledger	GET, POST /invoices; PATCH /invoices/{id}/paid; GET /recovery/priority; POST /recovery/reminder/{invoice_id}; GET, POST /bills
Customers	GET, POST /customers; GET /customers/{id}
Credit	GET /credit-score; GET /credit-score/history
Loan	GET /loan/readiness; GET /loan/recommendations; POST /loan/what-if
Reports	GET /reports/pnl; GET /reports/insights; GET /reports/export
Agent	POST /agent/chat (streaming)
Settings	GET, PUT /settings; POST, DELETE /lender-access

1.6 Request flow example

A user asks: "Will I run short on cash next week?"

1. The client sends POST /agent/chat.
2. The Agent Service sends the question and the tool schemas to the LLM.
3. The LLM requests get_cash_forecast(days=7).
4. The Agent Service calls the Forecast Engine with the business_id from the session; the engine reads transactions, invoices and bills.
5. The engine returns a shortage date and gap amount.
6. The Agent Service passes the result back to the LLM.
7. The LLM writes the answer, for example: "You may be short about 12,000 on Oct 22. Rahim Store owes you 15,000, 10 days overdue; collecting it covers the gap."

8. The answer streams back to the client.

1.7 Technology stack

Layer	Technology
Frontend	Next.js, TypeScript, Tailwind CSS, Recharts
Backend	Python, FastAPI, Pydantic, SQLAlchemy
Analytics	pandas, statsmodels or Prophet, scikit-learn
AI	Free-tier LLM APIs: Google Gemini (primary), Groq (fallback); Ollama local model as offline fallback; PaddleOCR for receipt scanning
Database	PostgreSQL
Deployment	Vercel (frontend), Railway or Render (backend), Supabase or Neon (database)

2. Core Algorithm Description

Six engines do the analysis. Each is deterministic and testable, and the AI agent only calls them as tools.

2.1 Cash flow prediction

Goal: project the daily cash balance for the next H days (H = 7 to 30) and flag shortages.

1. **Daily series.** Group transactions by day into income I(t) and expense E(t).
2. **Baseline forecast.** Use Holt-Winters exponential smoothing with weekly seasonality (period 7). With less than 60 days of history, fall back to a 28-day weighted moving average multiplied by a day-of-week factor.
3. **Known future events.** Add expected invoice inflows, weighted by each customer's historical on-time payment rate, and subtract bills that fall due.
4. **Balance projection.** Roll the balance forward day by day.
5. **Shortage detection.** Flag every day where the projected balance falls below the business's cash threshold.

$$\hat{I}(t) = \mathrm{WMA}_{28}(I) \cdot \mathrm{DOW}_I(t), \qquad \hat{E}(t) = \mathrm{WMA}_{28}(E) \cdot \mathrm{DOW}_E(t)$$

$$B(t) = B(t-1) + \hat{I}(t) - \hat{E}(t) + \sum_{\text{invoices due } t} a_i \, p_{\text{collect}}(c_i) - \sum_{\text{bills due } t} b_j$$

$$\text{Shortage at } t \iff B(t) < \text{threshold}, \qquad \text{Gap}(t) = \text{threshold} - B(t)$$

An 80% confidence band (±1.28 times the standard deviation of past forecast errors) is drawn around the line. A day is marked "at risk" when only the lower edge of the band crosses the threshold.

Output: daily projected balance, shortage dates, gap amounts and the main drivers of each shortage.

2.2 Smart payment recovery

Goal: rank customers by who should be contacted first, then draft a reminder for each.

Feature	Definition (normalized 0 to 1)	Weight
A	Outstanding amount ÷ largest outstanding amount	0.35
D	Days overdue ÷ 90, capped at 1	0.30
C	1 if collecting this amount closes a forecast shortage, else 0	0.20
L	Share of the customer's past invoices paid late	0.15

$$\text{Priority} = 0.35A + 0.30D + 0.20C + 0.15L$$

Reminder tone follows days overdue: friendly for 0 to 7 days, firm for 8 to 30 days, and a final notice after 30 days. The LLM fills a template with only the customer's name, amount, due date and tone; it sees no other customer data.

2.3 AI credit scoring (explainable)

Goal: a score from 300 to 900 with transparent reasons, computed over the last 6 to 12 months.

Factor	Definition (normalized 0 to 1)	Weight
f1	Average monthly income, log-scaled	0.20
f2	Income stability = 1 − coefficient of variation	0.15

Factor	Definition (normalized 0 to 1)	Weight
f3	Share of months with positive net cash flow	0.20
f4	1 − (outstanding receivables ÷ monthly income), clipped to 0 to 1	0.15
f5	On-time repayment rate	0.20
f6	Business age in months ÷ 60, capped at 1	0.10

$$S = 300 + 600 \sum_{i=1}^{6} w_i f_i$$

Explainability. Each factor's contribution is measured against a baseline b_i, the value for a typical healthy business. The three largest positive and three largest negative contributions are shown as reasons, for example: "High outstanding receivables lowered your score by 38 points."

$$\text{Contribution}_i = 600 \, w_i \, (f_i - b_i)$$

Score	Band
750 to 900	Excellent
650 to 749	Good
550 to 649	Fair
300 to 549	Needs improvement

Once real repayment outcomes are available, a gradient-boosted model (XGBoost) can replace the fixed weights, with SHAP values keeping each score explainable.

2.4 Loan readiness and recommendation

Goal: estimate a safe loan amount and matching products. The output is an estimate, not a loan approval.

1. **Conservative capacity.** Take the 25th percentile of monthly net cash flow over the last 12 months (NCF_p25), so one strong month cannot inflate capacity.
2. **Safe monthly installment.** Take a share α of that, set by credit band, minus existing installments: α = 0.40 (Excellent), 0.35 (Good), 0.25 (Fair), 0.15 (Needs improvement).
3. **Maximum principal.** Invert the standard EMI formula, with r the monthly interest rate and n the tenure in months.

4. **Product matching.** Keep products where the business's score meets min_score and P_max covers min_amount; rank by total cost of credit, lowest first.
5. **Readiness checklist.** Rule-based actions that would raise the score or capacity, such as "Collect receivables older than 30 days".

$$\text{SafeEMI} = \alpha \cdot \text{NCF}_{p25} - \text{ExistingEMI}$$

$$P_{\max} = \text{SafeEMI} \cdot \frac{(1+r)^n - 1}{r(1+r)^n}$$

2.5 What-if simulator

The user changes inputs, such as a new loan, a sales change of x%, or a new fixed expense, and the Forecast Engine re-runs with them. The original and scenario balance curves are shown side by side.

$$\hat{I}'(t) = \hat{I}(t)(1 + x), \qquad \hat{E}'(t) = \hat{E}(t) + \text{NewExpense} + \text{NewEMI}$$

2.6 Insight engine

Rule-based checks compare each expense category's current month with its 3-month average. A change over 20% raises an insight, which the LLM phrases for the dashboard summary.

2.7 AI financial agent

The agent runs a bounded tool-use loop of at most 5 steps. Its tools are get_cash_forecast, get_overdue_customers, get_credit_score, get_loan_recommendation, run_what_if and draft_reminder.

1. Send the system prompt, the user's question and the tool schemas to the LLM.
2. If the LLM requests a tool, validate the arguments, run the engine with the business_id from the authenticated session, and return the result to the LLM.
3. Repeat until the LLM answers in text, or stop at 5 steps with a fallback message.

Three grounding rules apply: every number in an answer must come from a tool result; the model can never choose the business_id; and all tools are read-only, so any action such as sending a reminder needs the user's confirmation in the UI.

3. Security Design

Every request is authenticated, every query is scoped to one business, financial data is encrypted in transit and at rest, and the AI agent can only read data through scoped, read-only tools.

3.1 Authentication

Control	Implementation
Password storage	bcrypt, cost factor 12
Access token	JWT, 15-minute lifetime
Refresh token	Rotating, 7-day lifetime, httpOnly secure SameSite cookie
Brute-force protection	Per-IP and per-account rate limits; temporary lockout after repeated failures
Second factor	Optional OTP by SMS or authenticator app

3.2 Authorization and tenant isolation

- **business_id scoping.** Every business-owned record has a business_id. Every query filters by the business_id in the authenticated token, never by one sent in the request.
- **Row-Level Security.** PostgreSQL RLS policies enforce the same rule in the database, as a second line of defence.
- **Object-level checks.** Any ID in a URL, such as /invoices/{id}, is checked for ownership before it is read or changed.
- **Roles.** owner (full access), staff (transactions and ledger only), lender (read-only, consent-scoped).

3.3 Data protection

Data	Protection
In transit	TLS 1.2 or later on every connection, HSTS
At rest	Database and storage encryption (AES-256)
Contact details	Phone numbers and emails encrypted at field level
Receipts and imports	Private storage buckets, short-lived signed URLs

Data	Protection
Backups	Encrypted, restricted access
Secrets	Free-tier LLM API keys (Gemini, Groq) and database credentials kept in backend environment secrets, never sent to the client

3.4 Consent-based lender access

Lenders see a business's score and reports only after the owner grants access. Each grant names its scope (which data), has an expiry date and can be revoked at any time. Every lender view is written to the audit log.

3.5 AI-specific security

Risk	Mitigation
Prompt injection through user data (a transaction note or receipt text)	User data is passed in clearly delimited data blocks; the system prompt tells the model to ignore instructions inside them
Cross-tenant data leak	Tools take business_id from the server session; the model cannot choose which business to query
Unauthorized actions	All agent tools are read-only; side effects such as sending a reminder need explicit user confirmation
Hallucinated figures	Numbers come only from engine outputs, and the answer is checked against the tool results
Over-sharing with the LLM	Only the fields a tool needs are sent; no passwords, tokens or full contact lists
Runaway loops and cost abuse	At most 5 tool steps per request, per-user rate limits and token budgets

Free-tier providers may keep or review submitted data, so only anonymized, minimal fields are sent to them (amounts, dates, category names; never names, phone numbers or account details). Owners can switch to the local Ollama model so no data leaves the server.

3.6 Application security

- **Input validation:** Pydantic schemas on every endpoint, with type, range and length checks.
- **SQL injection:** ORM with parameterized queries only.
- **Cross-site scripting:** React output escaping and a strict Content Security Policy.
- **CSRF:** SameSite cookies plus CSRF tokens on cookie-authenticated requests.
- **File uploads:** type and size limits, malware scanning, storage outside the web root.
- **Rate limiting:** per IP and per user on auth, AI and import endpoints.
- **Security headers:** HSTS, X-Content-Type-Options, X-Frame-Options, Referrer-Policy.

3.7 Monitoring and audit

The audit log records logins, data exports, lender access grants and views, and settings changes. Alerts fire on logins from unusual locations and on bulk data access. Application logs never contain passwords, tokens or full financial records.

3.8 Responsible AI and compliance

- Every credit score shows the factors behind it.
- Loan output is labelled "Estimate only, not a loan approval."
- Users can export or delete their data at any time.
- The design follows data-minimization and consent principles in line with GDPR-style practice and applicable local data-protection rules.

3.9 Threat summary

Threat	Controls
Account takeover	bcrypt, rate limits, optional second factor, short-lived tokens
Cross-business data access	business_id scoping, Row-Level Security, object-level checks
Data breach	Encryption in transit and at rest, field-level encryption
Prompt injection	Data and instruction separation, read-only tools
Unauthorized lender viewing	Explicit, scoped, expiring consent with audit log
Injection attacks	ORM, input validation, Content Security Policy

Threat	Controls
Abuse of AI endpoints	Rate limits, step limits, token budgets